UNIVERSIDAD MAYOR DE SAN ANDRÉS

Facultad de Ingeniería
Ingeniería Electrónica

BRAZO ROBOTICO DE 6 GRADOS DE LIBERTAD

ETN 1034: APLICACIÓN DE TECNICAS DE CONTROL

ETN 902: SISTEMAS DE CONTROL II

LABORATORIO NRO. 6

Herramientas para simulación, implementación y supervisión de procesos

ESTUDIANTES:

UNIV. BISMARK CHOQUE CHOQUE

UNIV. LUCAS DANIEL CONDARCO CALLE

UNIV. RODRIGO MIGUEL IPORRE COCA

DOCENTE:

ING. JORGE ANTONIO NAVA AMADOR

AUXILIARES:

UNIV. FABIAN MANUEL PLATA VARGAS

UNIV. LIMBER AJNOTA CAHUAYA

LA PAZ - BOLIVIA

LABORATORIO 6

HERRAMIENTAS PARA SIMULACIÓN, IMPLEMENTACIÓN Y SUPERVISIÓN DE PROCESOS

RESUMEN

El presente trabajo describe el desarrollo de un sistema de simulación, comunicación y supervisión aplicado al control de un brazo robótico de seis grados de libertad (6 GDL). El proyecto se desarrolló en tres etapas principales: validación del firmware mediante simulación en Proteus, implementación de la comunicación serial entre una computadora y un microcontrolador PIC16F1946, y desarrollo de una interfaz gráfica de usuario para supervisión y control utilizando Python.

La simulación permitió verificar el funcionamiento lógico del sistema antes de su implementación física. Posteriormente se estableció una comunicación serial bidireccional mediante un adaptador USB-Serial PL2303HX, logrando el intercambio de comandos entre la computadora y el microcontrolador. Finalmente, se desarrolló una interfaz hombre-máquina (HMI) que permitió controlar el brazo robótico desde una computadora personal.

Los resultados obtenidos demostraron la utilidad de las herramientas de simulación y supervisión para validar sistemas automatizados antes de su implementación definitiva, reduciendo tiempos de desarrollo y facilitando la detección de errores.

1. INTRODUCCIÓN

En los sistemas automatizados modernos, la simulación constituye una herramienta fundamental para validar diseños electrónicos y estrategias de control antes de su implementación física. La posibilidad de representar virtualmente un sistema permite reducir costos de desarrollo, identificar errores de programación y optimizar el comportamiento del sistema antes de interactuar con hardware real.

Dentro del ámbito de la automatización industrial, la integración de sistemas embebidos, protocolos de comunicación e interfaces de supervisión permite construir soluciones flexibles y escalables para el control de procesos.

En el presente laboratorio se desarrolló un sistema de control para un brazo robótico de seis grados de libertad utilizando un microcontrolador PIC16F1946. El trabajo incluyó la simulación del sistema en Proteus, la implementación de una comunicación serial entre la computadora y el sistema embebido, y el desarrollo de una interfaz gráfica en Python para supervisar y controlar el funcionamiento del brazo robótico.

2. OBJETIVOS

2.1 Objetivo General

Aplicar herramientas de simulación, implementación y supervisión de sistemas automatizados mediante el desarrollo de un sistema de control para un brazo robótico de seis grados de libertad, integrando un entorno de simulación en Proteus, un sistema embebido basado en el microcontrolador PIC16F1946 y una interfaz de supervisión desarrollada en Python, con el propósito de validar la comunicación, el funcionamiento del sistema y la interacción entre hardware y software antes de su implementación definitiva.

2.2 Objetivos Específicos

- Diseñar y validar un entorno de simulación en Proteus que permita verificar el funcionamiento del firmware implementado en el microcontrolador PIC16F1946 y analizar el comportamiento general del sistema de control del brazo robótico.
- Implementar la comunicación serial UART entre una computadora personal y el microcontrolador, utilizando un adaptador USB-Serial PL2303HX para garantizar el intercambio bidireccional de información.
- Desarrollar una interfaz hombre-máquina (HMI) en Python que permita supervisar el estado de la comunicación y controlar las funciones principales del brazo robótico desde una computadora.
- Verificar la integración entre el entorno de simulación, el sistema embebido y la interfaz de supervisión, evaluando la respuesta del sistema ante diferentes comandos de control.
- Analizar los resultados obtenidos durante las etapas de simulación e implementación física, identificando las ventajas de la simulación como herramienta de validación previa y las limitaciones existentes respecto al comportamiento del sistema real.
- Comparar el comportamiento observado durante la simulación con el funcionamiento obtenido en la implementación física, evaluando la correspondencia entre ambos entornos de trabajo.

2. PROCEDIMIENTO DE LABORATORIO

El desarrollo del proyecto se realizó siguiendo la metodología planteada en la guía de laboratorio, adaptando cada una de las etapas al diseño e implementación del sistema de control para un brazo robótico de seis grados de libertad basado en un microcontrolador PIC16F1946.

2.1 Investigación Teórica y Familiarización con Herramientas de Simulación

2.1.1. Conceptos Teóricos

Interfaces Hombre-Máquina (HMI)

Una HMI (Human-Machine Interface) es el panel o pantalla que permite a un usuario humano interactuar con un sistema automatizado o máquina. Su función teórica es doble:

- **Traducir Datos Complexos:** Transforma las variables eléctricas y digitales del sistema embebido (voltajes, registros, variables) en gráficos, texto o alarmas visuales fáciles de entender.
- **Envío de Comandos:** Permite al operador ingresar parámetros o detener procesos, traduciendo acciones del usuario en señales digitales para el microcontrolador.

Comunicación Serial UART

La UART (Universal Asynchronous Receiver-Transmitter) no es un protocolo de bus físico en sí, sino un componente o circuito físico encargado de la comunicación serial asíncrona.

- **Asíncrona:** Significa que no utiliza una señal de reloj común para sincronizar el emisor y el receptor. En su lugar, se configuran de antemano con la misma velocidad de transmisión (baudios).
- **Mecanismo:** Utiliza solo dos líneas de datos principal: Transmisión (**TX**) y Recepción (**RX**). Los datos se empaquetan en tramas que incluyen un bit de inicio (start bit), de 5 a 9 bits de datos, un bit de paridad opcional (para detectar errores) y uno o dos bits de parada (stop bits).

2.1.2. Herramientas de Software Seleccionadas

Simulador Proteus (ISIS)

Proteus es un entorno de diseño electrónico y simulación que combina la captura de esquemáticos con la simulación de circuitos analógicos y digitales. Su ventaja teórica principal es la **Co-simulación VSM** (Virtual System Modelling):

- Permite cargar el archivo binario compilado (código máquina) directamente en el modelo virtual del microcontrolador.
- El software emula en tiempo real los ciclos de instrucción del procesador a la par que calcula las leyes físicas de los componentes analógicos (resistencias, LEDs, motores) conectados a sus pines.

Compilador HI-TECH C

Es un compilador especializado para la arquitectura de microcontroladores de Microchip. Su rol teórico es la traducción de niveles de abstracción:

- Transforma el código fuente escrito en **C** (lenguaje de alto nivel accesible para humanos) en **código de máquina o hexadecimal** (instrucciones binarias que el hardware del PIC puede ejecutar).
- Optimiza el uso de la memoria limitada del microcontrolador (Flash y RAM) mediante la gestión eficiente de los bancos de registros y la pila (stack).

Lenguaje Python (para HMI)

Python es un lenguaje de programación interpretado de alto nivel y multipropósito. En el contexto de las HMI, teóricamente destaca por su capacidad de abstracción y modularidad:

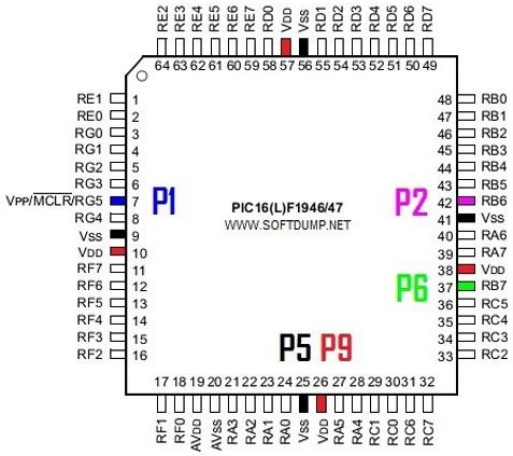
- Permite gestionar la comunicación con el puerto serial mediante hilos de ejecución (threading) para recibir datos en segundo plano sin congelar la pantalla.
- Facilita el diseño de interfaces de usuario mediante librerías gráficas (como Tkinter o PyQt) y el procesamiento rápido de datos gracias a su manejo dinámico de tipos de datos.

2.1.3. Componentes de Hardware Analizados

Microcontrolador PIC16F1946 y módulo EUSART

El PIC16F1946 es un circuito integrado de 8 bits de rango medio que ejecuta instrucciones basadas en la arquitectura Harvard (memorias de datos y de programa separadas).

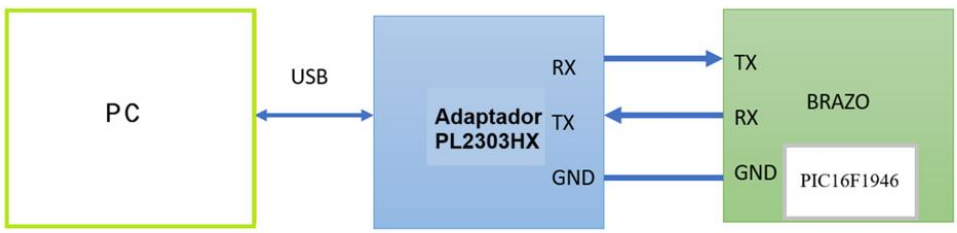
- **Módulo EUSART:** El módulo UART en el microcontrolador PIC16F1946 se denomina formalmente EUSART (Transmisor Receptor Síncrono Asíncrono Universal Mejorado). Es un periférico de comunicación serial diseñado para el intercambio de datos bidireccional y de baja velocidad entre el microcontrolador y otros dispositivos como ordenadores, módulos Bluetooth, GPS, pantallas inteligentes u otros microcontroladores.
- El PIC16F1946 cuenta con dos módulos EUSART idénticos e independientes (EUSART1 y EUSART2), lo que te permite gestionar dos canales de comunicación serial de forma simultánea



Adaptador USB-Serial PL2303HX

Las computadoras modernas ya no incluyen puertos seriales nativos (RS-232), sino puertos USB de alta velocidad. El PL2303HX es un puente de hardware que soluciona esta incompatibilidad mediante la conversión de protocolos:

- **Lado PC (USB):** Se comporta como un dispositivo de clase CDC (Communications Device Class), permitiendo que el sistema operativo de la computadora simule un puerto COM virtual.
- **Lado Embebido (TTL):** Traduce los paquetes de datos USB a niveles lógicos de voltaje serial directos (generalmente 0V y 5V/3.3V) compatibles con los pines TX y RX del microcontrolador PIC.

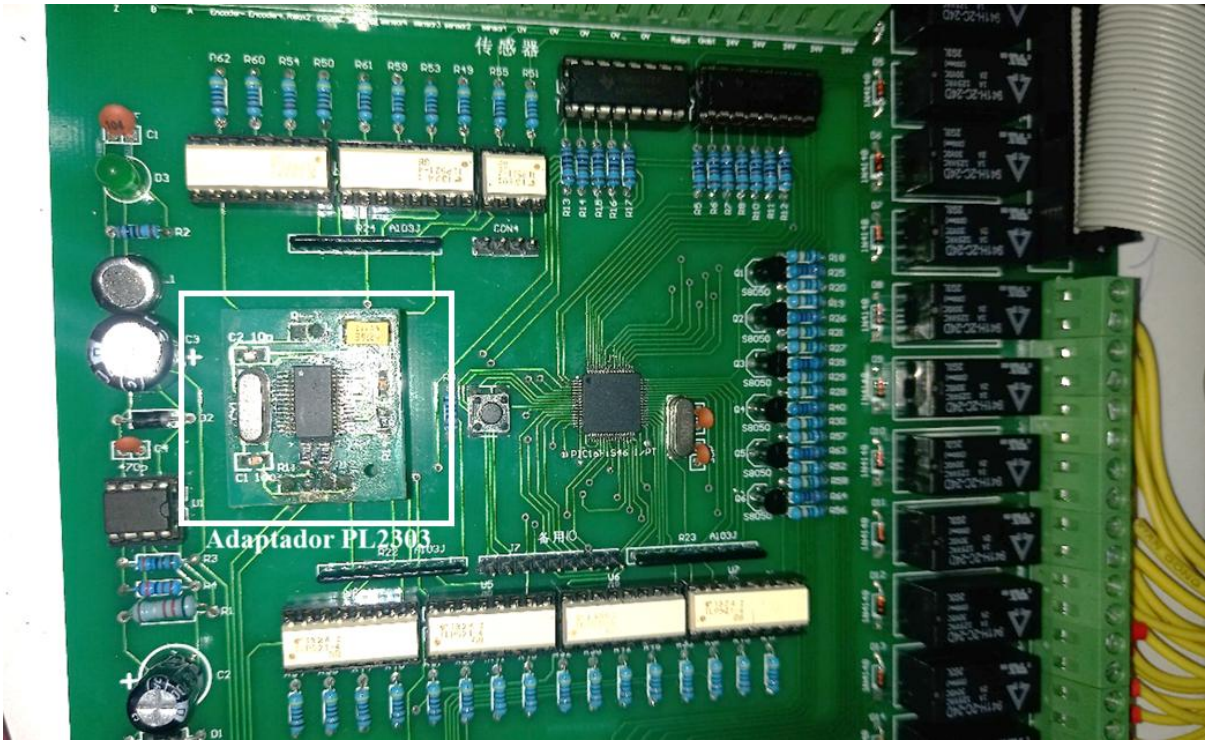


2.2 Configuración del Entorno de Simulación

2.2.1. Construcción del Entorno de Simulación

El diseño del esquema electrónico en Proteus consistió en la interconexión virtual de los siguientes bloques fundamentales:

- **Microcontrolador Principal:** Ubicación del **PIC16F1946** como cerebro del sistema de control.
- **Circuitos Auxiliares:** Implementación de la electrónica mínima para que el integrado funcione. Esto incluye el circuito de reinicio (Master Clear o **MCLR**) mediante una resistencia de pull-up, y las líneas de alimentación de corriente continua.
- Para desarrollar una configuración correcta se realizó la verificación de la implementación física del adaptador USB-Serial PL2303HX conectado entre la computadora y el microcontrolador PIC16F1946.



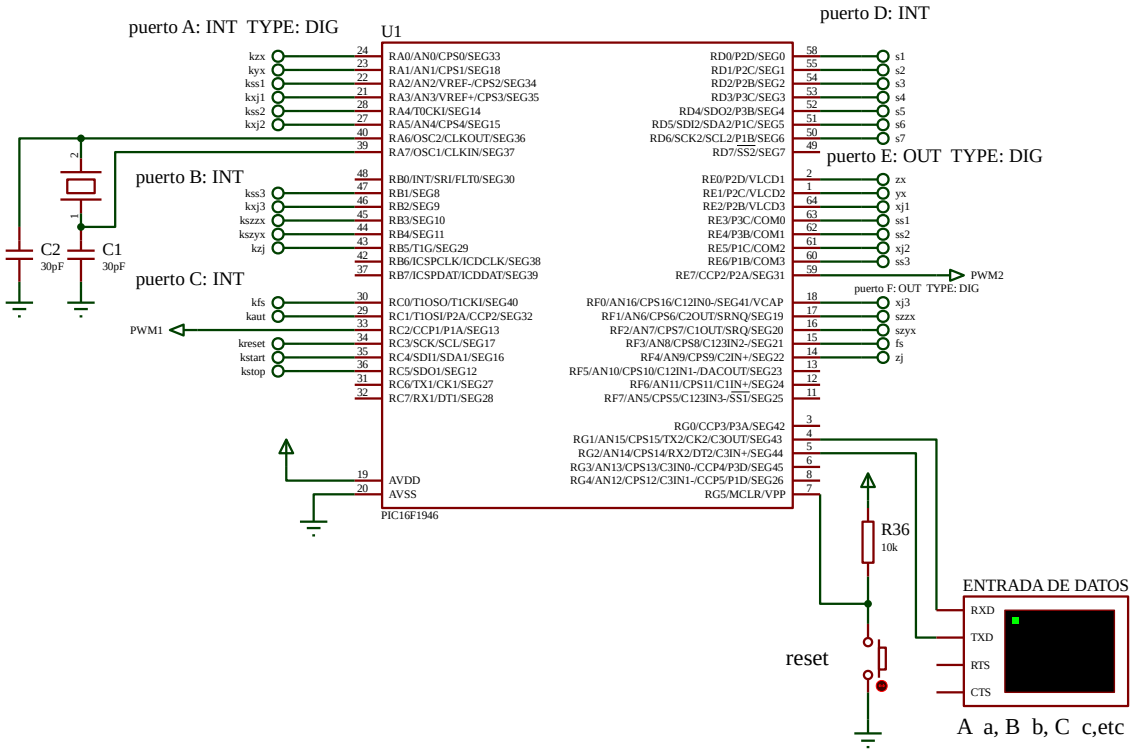
Se observo que el adaptador **PL2303** esté conectado a los pines 4 y 5 del microcontrolador.

En el encapsulado TQFP de 64 pines del PIC16F1946, los pines físicos corresponden exactamente a lo siguiente:

- **Pin 4:** Corresponde al puerto **RG1**, que tiene asignada la función periférica **TX2**

- **Pin 5:** Corresponde al puerto **RG2**, que tiene asignada la función periférica **RX2**

Por lo tanto, el firmware del PIC se debe configurar para transferir datos a través del segundo módulo de comunicación serie mejorada (EUSART2).

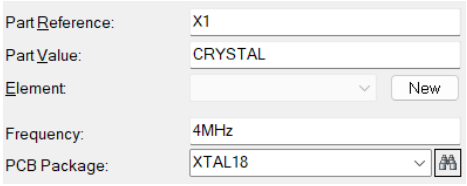


2.2.2. Configuración de los Elementos Principales

Oscilador Principal del Sistema

El oscilador es el metrónomo del microcontrolador; dicta la velocidad con la que se ejecutan las instrucciones del firmware.

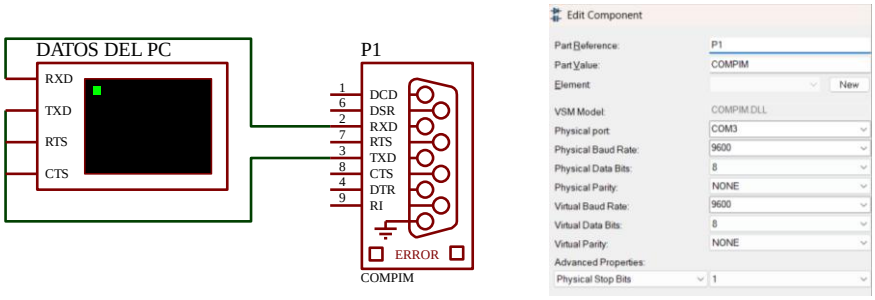
- **Configuración:** Teóricamente, se define si se utiliza un cristal de cuarzo externo (modo XT o HS) o el oscilador interno del PIC.
- **Importancia:** Su correcta declaración en los registros de configuración (*Configuration Bits*) es crítica. Si la frecuencia del oscilador configurada en el software no coincide con la velocidad real del simulador, los temporizadores internos fallarán y la velocidad de la comunicación serial se desincronizará por completo.



Módulo de Comunicación Serial Virtual

Para validar la UART sin una computadora física conectada, Proteus utiliza herramientas de instrumentación virtual.

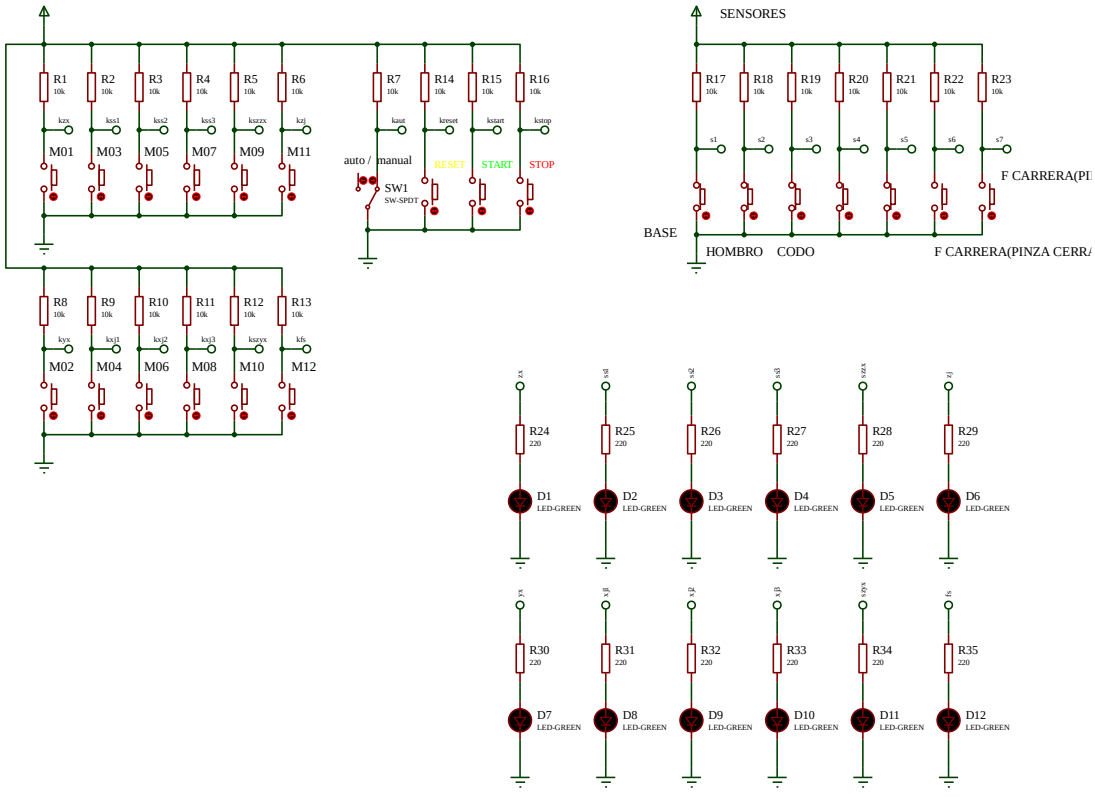
- **Terminal Virtual (Virtual Terminal):** Herramienta que se conecta directamente a los pines **TX** (transmisión) y **RX** (recepción) del PIC16F1946. Permite visualizar en pantalla los caracteres ASCII que envía el chip y simular la escritura de comandos desde un teclado hacia el microcontrolador.
- **COMPIM:** En caso de requerir interactuar con software externo (como el script de Python), se configura este componente que mapea el puerto serial del simulador con un puerto COM virtual del sistema operativo.



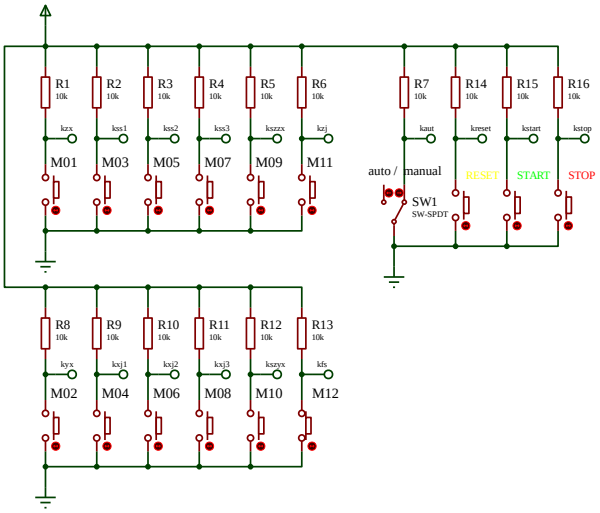
Para simular se necetita un COM virtual

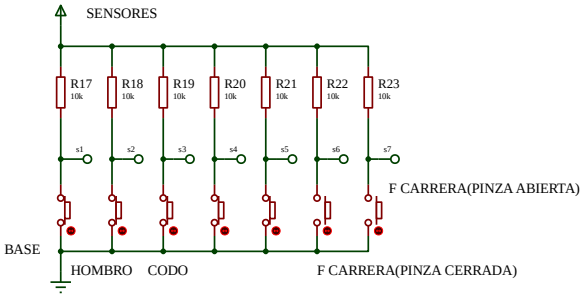
Entradas y Salidas Digitales (I/O)

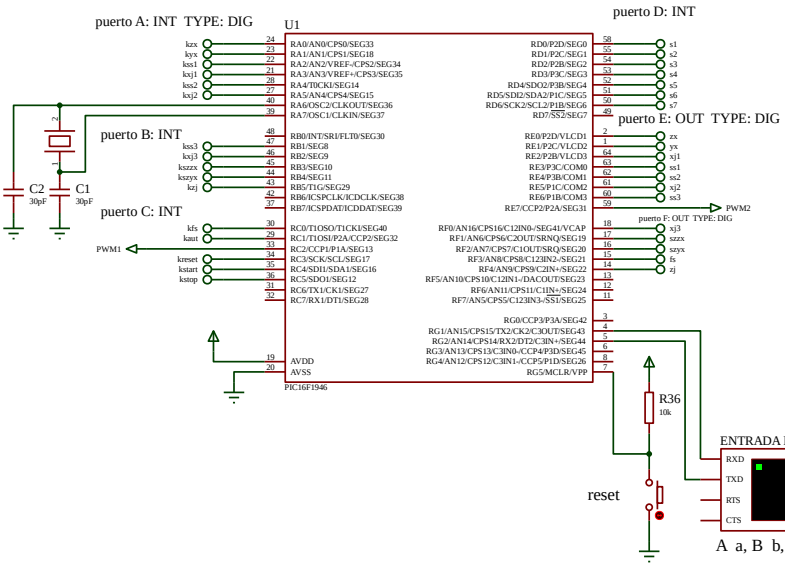
- **Entradas:** Configuración de pines para recibir señales externas (los pulsadores en el tablero de control).
- **Salidas:** Configuración de pines para enviar órdenes (las dos señales PWM, y los pines

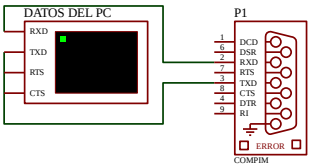


2.2.3. Circuito simulado

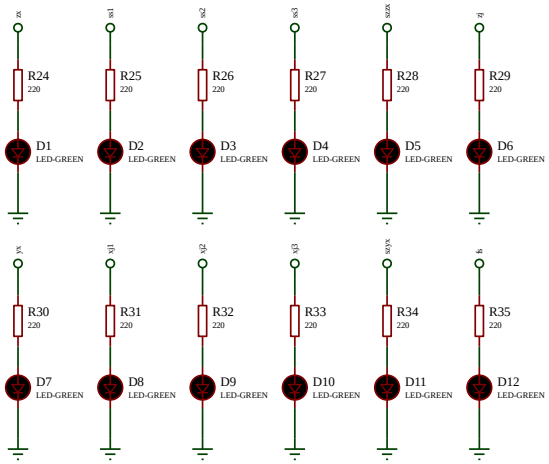








Para simular se necetita un COM virtual



2.3. Configuración e Implementación de la Comunicación entre la Computadora y el PIC16F1946

2.3.1 Selección del protocolo de comunicación

Con el objetivo de establecer un enlace de comunicación entre la computadora personal y el sistema de control del brazo robótico, se seleccionó el protocolo UART (Universal Asynchronous Receiver Transmitter) como mecanismo principal para el intercambio de información.

La elección de este protocolo se realizó debido a las siguientes ventajas:

- Bajo costo de implementación.
- Facilidad de programación.
- Baja utilización de recursos del microcontrolador.
- Amplia compatibilidad con dispositivos USB-Serial.
- Facilidad de integración con aplicaciones desarrolladas en Python.

La comunicación UART permitió transmitir comandos desde la computadora hacia el microcontrolador, los cuales posteriormente fueron interpretados para controlar los diferentes movimientos del brazo robótico.

2.3.2 Configuración del hardware de comunicación

Para establecer la comunicación física se utilizó un adaptador USB-Serial basado en el circuito integrado PL2303HX.

Este dispositivo permitió convertir la interfaz USB de la computadora en una interfaz UART compatible con el microcontrolador PIC16F1946.

Las conexiones implementadas fueron las siguientes:

Adaptador PL2303HX	**PIC16F1946**	**Función**
TXD	RG2 (RX2)	Recepción de datos
RXD	RG1 (TX2)	Transmisión de datos
GND	GND	Referencia común

El intercambio cruzado entre las líneas TX y RX permitió establecer la comunicación bidireccional entre ambos dispositivos.

Asimismo, la conexión a tierra común garantizó la correcta referencia eléctrica necesaria para una transmisión estable.

2.3.3 Configuración del módulo EUSART2 del PIC16F1946

El microcontrolador PIC16F1946 dispone de un módulo EUSART, el cual fue configurado para operar en modo asíncrono.

Inicialmente se deshabilitó la función analógica de los pines utilizados para la comunicación mediante la instrucción:

```
ANSELG = 0x00;
```

Posteriormente se configuró la dirección de los pines:

```
TRISG1 = 0;
TRISG2 = 1;
```

donde: RG1 se configuró como salida (TX2) y RG2 se configuró como entrada (RX2).

A continuación, se establecieron los parámetros principales de funcionamiento.

- Configuración del modo asíncrono: `TX2STAbits.SYNC = 0;`

Esta instrucción habilitó el funcionamiento en modo asíncrono.

- Configuración del tamaño de palabra: `TX2STAbits.TX9 = 0;`

Se seleccionó una transmisión de 8 bits.

- Configuración de alta velocidad: `TX2STAbits.BRGH = 1;`

Se habilitó el modo de alta velocidad para el generador de baudios.

- Configuración del modo de 8 bits del generador de baudios:

```
BAUD2CONbits.BRG16 = 0;
```

El generador de baudios fue configurado en modo de 8 bits.

- Configuración de la velocidad de transmisión:

```
SP2BRGH = 0;
SP2BRGL = 103;
```

Este valor fue seleccionado experimentalmente para obtener una velocidad de transmisión cercana a 9600 baudios utilizando una frecuencia de oscilador de 4 MHz.

2.3.4 Habilitación de la transmisión y recepción

Una vez configurado el módulo UART, se habilitaron las funciones necesarias para iniciar la comunicación.

La interfaz serial fue habilitada mediante: `RC2STAbits.SPEN = 1;`

Posteriormente se habilitó la recepción continua: `RC2STAbits.CREN = 1;`

Finalmente se activó el transmisor: `TX2STAbits.TXEN = 1;`

Estas configuraciones permitieron que el microcontrolador quedara preparado para transmitir y recibir datos de forma permanente.

2.3.5 Desarrollo de las rutinas de transmisión

Para transmitir información hacia la computadora se desarrolló la función `UART2_Write()`.

```
void UART2_Write(char dato)
{
    while(!TX2IF);

    TX2REG = dato;
}
```

Esta rutina espera a que el buffer de transmisión se encuentre disponible antes de enviar un nuevo carácter.

Posteriormente se desarrolló una función auxiliar denominada `UART2_Text()`, cuya finalidad consiste en transmitir cadenas completas de texto.

```
void UART2_Text(const char *txt)
{
    while(*txt)
    {
        UART2_Write(*txt++);
    }
}
```

Esta función fue utilizada para enviar mensajes informativos durante la inicialización del sistema.

Como prueba de funcionamiento se implementó el siguiente mensaje:

```
UART2_Text("BRAZO LISTO\r\n");
```

La recepción correcta de este mensaje confirmó que la comunicación había sido establecida satisfactoriamente.

2.3.6 Desarrollo del sistema de recepción y procesamiento de comandos

Para controlar el brazo robótico se desarrolló una rutina denominada `PC_Control()`.

Esta función verifica permanentemente la existencia de nuevos datos recibidos desde la computadora.

```
if(RC2IF)
{
    cmd = RC2REG;
}
```

Una vez recibido un carácter, el programa lo interpreta mediante una estructura switch.

Cada comando activa una variable interna asociada a un movimiento específico del brazo robótico.

Por ejemplo:

```
case 'A':
    pc_zx = 1;
    break;

case 'a':
    pc_zx = 0;
    break;
```

Las letras mayúsculas activan un movimiento y las letras minúsculas lo desactivan.

Los comandos implementados fueron:

Comando	**Función**
A / a	Activar o desactivar ZX
B / b	Activar o desactivar YX
C / c	Activar o desactivar SS1
D / d	Activar o desactivar XJ1
E / e	Activar o desactivar SS2
F / f	Activar o desactivar XJ2
G / g	Activar o desactivar SS3
H / h	Activar o desactivar XJ3
I / i	Activar o desactivar SZZX
J / j	Activar o desactivar SZYX
K / k	Activar o desactivar ZJ
L / l	Activar o desactivar FS
S	Detener todos los movimientos

2.3.7 Implementación del sistema de protección contra errores

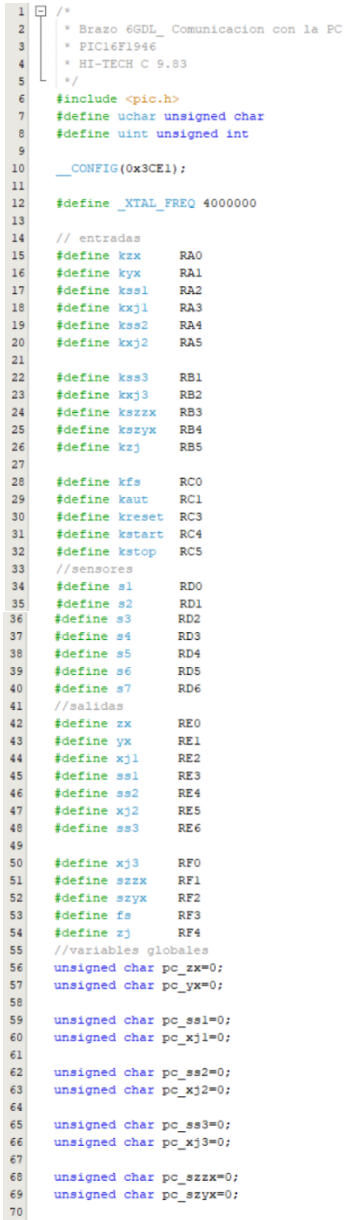
Durante las pruebas experimentales se identificó la posibilidad de que ocurrieran errores de sobrecarga en el buffer de recepción.

Para solucionar este problema se implementó un mecanismo de recuperación automática mediante el control del bit `OERR`.

```
if(RC2STAbits.OERR)
{
    RC2STAbits.CREN = 0;
    RC2STAbits.CREN = 1;
}
```

Esta estrategia permitió restablecer automáticamente la recepción sin necesidad de reiniciar el sistema.

2.3.8. Código de Conexión de la PC con el PIC en lenguaje C



```
/*
 * Brazo 6GDL_ Comunicacion con la PC
 * PIC16F1946
 * HI-TECH C 9.83
 */
#include <pic.h>
#define uchar unsigned char
#define uint unsigned int

__CONFIG(0x3CE1);

#define _XTAL_FREQ 4000000

// entradas
#define kzx     RA0
#define kyx     RA1
#define kss1    RA2
#define kxj1    RA3
#define kss2    RA4
#define kxj2    RA5

#define kss3    RB1
#define kxj3    RB2
#define kszzx   RB3
#define kszyx   RB4
#define kzj     RB5

#define kfs     RC0
#define kaut    RC1
#define kreset  RC3
#define kstart  RC4
#define kstop   RC5
//sensores
#define s1      RD0
#define s2      RD1
#define s3      RD2
#define s4      RD3
#define s5      RD4
#define s6      RD5
#define s7      RD6
//salidas
#define zx      RE0
#define yx      RE1
#define xj1     RE2
#define ss1     RE3
#define ss2     RE4
#define xj2     RE5
#define ss3     RE6

#define xj3     RF0
#define szzx    RF1
#define szyx    RF2
#define fs      RF3
#define zj      RF4
//variables globales
unsigned char pc_zx=0;
unsigned char pc_yx=0;

unsigned char pc_ss1=0;
unsigned char pc_xj1=0;

unsigned char pc_ss2=0;
unsigned char pc_xj2=0;

unsigned char pc_ss3=0;
unsigned char pc_xj3=0;

unsigned char pc_szzx=0;
unsigned char pc_szyx=0;
```

Se incluye la librería principal del microcontrolador PIC.

Además, se crean alias para simplificar la escritura del código:

- `uchar`: representa un entero sin signo de 8 bits.
- `uint`: representa un entero sin signo de 16 bits.

Se asignan nombres descriptivos a las entradas físicas del microcontrolador.

Estas entradas corresponden a:

- Pulsadores manuales.
- Botones de inicio y parada.
- Selector de modo automático.
- Comandos de movimiento.

```
unsigned char pc_zj=0;
unsigned char pc_fs=0;
//-----------------------------------------------
// UART
//-----------------------------------------------
void UART2_Write(char dato)
{
    while(!TX2IF);
    TX2REG = dato;
}

void UART2_Text(const char *txt)
{
    while(*txt)
    {
        UART2_Write(*txt++);
    }
}

void UART2_Init(void)
{
    ANSELG = 0x00;

    TRISG1 = 0;      // TX2
    TRISG2 = 1;      // RX2

    TX2STAbits.SYNC = 0;
    TX2STAbits.TX9  = 0;
    TX2STAbits.BRGH = 1;

    BAUD2CONbits.BRG16 = 0;

    SP2BRGH = 0;
    SP2BRGL = 103;      // 25

    RC2STAbits.SPEN = 1;
    RC2STAbits.CREN = 1;

    TX2STAbits.TXEN = 1;
}
//-----------------------------------------------
// PWM
//-----------------------------------------------
void PWM_init(void)
{
    PR2 = 50;
    CCP1CON = 0x0F;
    CCPR1L = 25;
    CCPTMRS0 = 0x00;
    T2CON = 0x07;
    TRISC2 = 0;

    PR4 = 50;
    CCP2CON = 0x0F;
    CCPR2L = 25;
    CCPTMRS0 = 0x04;
    T4CON = 0x07;
    APFCON = 0x04;
    TRISE7 = 0;
}
//-----------------------------------------------
// PUERTOS
//-----------------------------------------------
void IO_init(void)
{
    ANSELA = 0x00;
    ANSELE = 0x00;
    ANSELF = 0x00;
    ANSELG = 0x00;

    TRISA = 0xFF;
    TRISB = 0xFF;
    TRISD = 0xFF;

    TRISC0 = 1;
    TRISC1 = 1;
    TRISC3 = 1;
    TRISC4 = 1;
    TRISC5 = 1;

    TRISE = 0x00;
    TRISF = 0x00;

    PORTE = 0x00;
    PORTF = 0x00;
}
//-----------------------------------------------
// CONTROL DESDE PC
//-----------------------------------------------
void PC_Control(void)
{
    char cmd;

    if(RC2STAbits.OERR)
    {
        RC2STAbits.CREN = 0;
        RC2STAbits.CREN = 1;
    }

    if(RC2IF)
    {
        cmd = RC2REG;

        switch(cmd)
        {
```

`UART2_Write()`

Esta función transmite un carácter a través del puerto serial UART.

Antes de enviar el dato, espera a que el transmisor esté disponible.

`UART2_Text()`

Esta función envía una cadena completa de caracteres.

Se utiliza para enviar mensajes de estado hacia la computadora.

`PWM_init()`

Esta función inicializa dos módulos PWM.

Se configuran:
- Periodo de trabajo.
- Ciclo útil inicial.
- Temporizadores asociados.

La señal PWM permite controlar la velocidad o posición de algunos actuadores.

`void IO_init(void)`

Esta función configura todos los puertos del microcontrolador.

Se realizan las siguientes tareas:

- Desactivar entradas analógicas.
- Configurar entradas digitales.
- Configurar salidas digitales.
- Inicializar las salidas en cero.

Esto asegura un estado conocido al iniciar el sistema.

void PC_Control(void)

Esta función recibe comandos enviados desde la PC mediante UART.

Cada letra activa o desactiva un movimiento.

Ejemplo:

'A' : activar zx
'a' : desactivar zx

El comando: 'S' : detiene completamente el brazo robótico.

```
        case 'A': pc_zx=1; break;
        case 'a': pc_zx=0; break;
        case 'B': pc_yx=1; break;
        case 'b': pc_yx=0; break;
        case 'C': pc_ss1=1; break;
        case 'c': pc_ss1=0; break;
        case 'D': pc_xj1=1; break;
        case 'd': pc_xj1=0; break;
        case 'E': pc_ss2=1; break;
        case 'e': pc_ss2=0; break;
        case 'F': pc_xj2=1; break;
        case 'f': pc_xj2=0; break;
        case 'G': pc_ss3=1; break;
        case 'g': pc_ss3=0; break;
        case 'H': pc_xj3=1; break;
        case 'h': pc_xj3=0; break;
        case 'I': pc_szzx=1; break;
        case 'i': pc_szzx=0; break;
        case 'J': pc_szyx=1; break;
        case 'j': pc_szyx=0; break;
        case 'K': pc_zj=1; break;
        case 'k': pc_zj=0; break;
        case 'L': pc_fs=1; break;
        case 'l': pc_fs=0; break;
        case 'S':
                  pc_zx=0;
                  pc_yx=0;
                  pc_ss1=0;
                  pc_xj1=0;
                  pc_ss2=0;
                  pc_xj2=0;
                  pc_ss3=0;
                  pc_xj3=0;
                  pc_szzx=0;
                  pc_szyx=0;
                  pc_zj=0;
                  pc_fs=0;
    PORTE=0;
    PORTF=0;
    break;
        }
    }
}

void Manual_Control(void)
{
    zx   = ((kzx==0)   || pc_zx) && (s1==0);
    yx   = ((kyx==0)   || pc_yx) && (s1==0);

    ss1  = ((kss1==0)  || pc_ss1) && (s2==0);
    xj1  = ((kxj1==0)  || pc_xj1) && (s2==0);

    ss2  = ((kss2==0)  || pc_ss2) && (s3==0);
    xj2  = ((kxj2==0)  || pc_xj2) && (s3==0);

    ss3  = ((kss3==0)  || pc_ss3) && (s4==0);
    xj3  = ((kxj3==0)  || pc_xj3) && (s4==0);

    szzx = ((kszzx==0) || pc_szzx) && (s5==0);
    szyx = ((kszyx==0) || pc_szyx) && (s5==0);

    zj   = ((kzj==0)   || pc_zj) && (s6==1);
    fs   = ((kfs==0)   || pc_fs) && (s7==1);
}
//-----------------------------------------------
// Programa principal
//-----------------------------------------------
void main(void)
{
    IO_init();

    PWM_init();

    UART2_Init();

    UART2_Text("BRAZO LISTO\r\n");

    while(1)
{
    PC_Control();
    if(kstop == 0)
    {
        pc_zx=0;
        pc_yx=0;
        pc_ss1=0;
        pc_xj1=0;
        pc_ss2=0;
        pc_xj2=0;
        pc_ss3=0;
        pc_xj3=0;
        pc_szzx=0;
        pc_szyx=0;
        pc_zj=0;
        pc_fs=0;

        PORTE=0;
        PORTF=0;
    }

    if(kaut == 0)
    {
        Manual_Control();
    }
}
}
```

`void Manual_Control(void)`

Esta función combina:

- Pulsadores físicos.
- Órdenes recibidas desde la PC.
- Señales de los sensores.

La salida solo se activa si:

1. El usuario presiona el pulsador o la PC envía la orden.
2. El sensor de seguridad lo permite.

Ejemplo:

```
zx = ((kzx==0) || pc_zx) && (s1==0);
```

Esto significa:

- Activar el movimiento zx si el botón está presionado o la PC lo ordena.
- Solo si el sensor s1 no está activado.

`void main(void)`

Es la función principal del sistema.

Inicializa:

```
IO_init();
```

```
PWM_init();
```

```
UART2_Init();
```

Posteriormente envía el mensaje: BRAZO LISTO

indicando que el sistema está preparado.

Finalmente ejecuta un ciclo infinito:

`while(1)`

Dentro del ciclo se realizan tres tareas:

1. Escuchar órdenes de la PC: `PC_Control();`

2. Botón de parada: `if(kstop == 0)`

Detiene todos los movimientos y reinicia las variables.

3. Modo manual: `if(kaut == 0){Manual_Control();}`

Permite controlar el brazo mediante los pulsadores y las órdenes recibidas desde la PC.

2.3.8 Integración con la interfaz desarrollada en Python

Una vez verificada la comunicación serial, se integró el sistema con la interfaz gráfica desarrollada en Python.

Para la comunicación entre el brazo de 6 grados de libertad y un computador, por la parte del computador elegimos el lenguaje de Python para establecer la comunicación, lo que haremos es establecer comunicación con el microcontrolador mediante el modulo PL-2303HX el cual nos brindara la conexión usb entre el microcontrolador y el computador.

ESTRUCTURA DEL PROGRAMA:

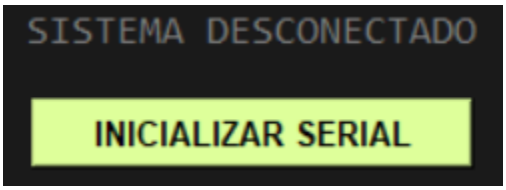
a) COMUNICACIÓN

```
# --- FUNCIONES ---
def conectar():
    global ser
    try:
        if ser and ser.is_open: ser.close()
        ser = serial.Serial(PUERTO, BAUDIOS, timeout=1)
        lbl_estado.config(text=f"SISTEMA ONLINE: {PUERTO}", fg="#00FF00")
    except Exception as e:
        lbl_estado.config(text="ERROR DE CONEXIÓN", fg="#FF4444")

def enviar(cmd):
    if ser and ser.is_open:
        ser.write(cmd.encode())

def cerrar_app():
    if ser and ser.is_open: ser.close()
    root.destroy()
```

En esta parte del código es donde transformamos la acción física del operador en una señal digital (o trama de envió) que podamos transmitir de nuestro computador al microcontrolador y que este sea capaz de recepcionar dicha trama, estructuramos esta primera parte en 3 subpartes:



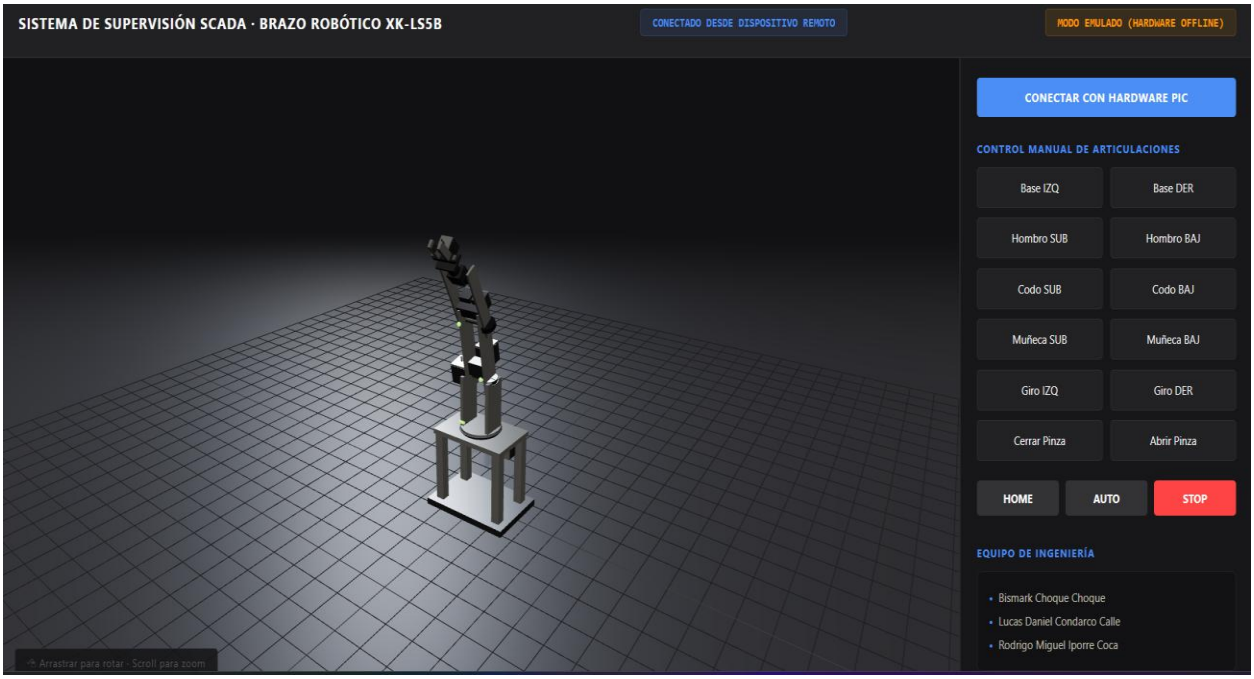
- Gestión de Sesión: Inicialmente debemos establecer la comunicación con el microcontrolador para esto inicialmente presionamos el botón “INICIALIZAR SERIAL”, este abrirá un puerto de comunicación bidireccional el cual nos ayuda a establecer la comunicación de Python, ya que por defecto el sistema operativo del computador puede bloquear este acceso.

Se establecio una medida de seguridad por si en algún descuido se abre la sesión 2 veces y ya tiene la comunicación establecida, el programa cerrara la sesión anterior y conectara la sesión actual.

En “ser = serial.Serial(PUERTO, BAUDIOS, timeout=1)” es donde ocurre la conexión, se abre el COM configurado a la velocidad que definimos, en nuestro caso 9600 baudios.

- Traducción: Realizamos el envío y recepción en código UTF-8/ASCII, por lo que en nuestro programa convierte nuestras variables asignadas al código UTF-8/ASCII mediante la función “enviar(cmd)”, al enviar un carácter este primero pasa por la función la cual atraves de “ser.write(cmd.encode())” dicho código convierte el carácter al código UTF-8/ASCII el cual es un numero binario de 8 bits. Esta es la secuencia de bits que enviaremos a través del USB al microcontrolador, y de la misma forma en el microcontrolador establecimos el mismo formato de envio, por lo que la recepción del microcontrolador al computador será en UTF-8/ASCII.
- Seguridad: Al interceptar el cierre de la interfaz, ejecutamos “ser.close()”. Esto le envía una señal al sistema operativo para que libere el hardware, de no hacerlo se puede reducir la vida útil del hardware y podrían aparecer errores a futuro.

b) INTERFAZ GRAFICA



Para la interfaz gráfica hicimos uso de la librería “tkinter” con un estilo oscuro para reducir la fatiga visual, la interfaz nos muestra información relevante y la parte de acción o donde irán los botones.

- Encabezado y Sidebar: Utilizan contenedores “Frame” para organizar la identidad del proyecto (Logo UMSA) y el equipo de desarrollo.

```python
# --- VENTANA PRINCIPAL ---
root = tk.Tk()
root.title("Robotic Link - Control System")
root.geometry("1000x700")
root.configure(bg="#1a1a1a")
root.protocol("WM_DELETE_WINDOW", cerrar_app)

# --- ENCABEZADO ---
header = tk.Frame(root, bg="#333333", height=80)
header.pack(fill="x")

try:
    ruta_logo = r"C:\Users\IPORRE\Desktop\Prueba_Brazo_Comunicacion\logo1.png"
    img_logo = Image.open(ruta_logo).resize((100, 100), Image.Resampling.LANCZOS)
    logo_tk = ImageTk.PhotoImage(img_logo)
    lbl_logo = tk.Label(header, image=logo_tk, bg="#333333")
    lbl_logo.image = logo_tk
    lbl_logo.pack(side="left", padx=20)
except Exception as e:
    print(f"Logo no encontrado: {e}")

tk.Label(header, text="CONTROL DE BRAZO  DE 6 GRADOS DE LIBERTAD V2.0",
         fg="white", bg="#333333", font=("Orbitron", 18, "bold")).pack(side="left")

# --- PANEL LATERAL ---
sidebar = tk.Frame(root, bg="#252525", width=250)
sidebar.pack(side="right", fill="y", padx=10, pady=10)

tk.Label(sidebar, text="EQUIPO DE DESARROLLO", fg="#deff9a",
         bg="#252525", font=("Arial", 12, "bold")).pack(pady=10)

integrantes = ["BISMARK CHOQUE CHOQUE", "LUCAS CONDARCO CALLE", "RODRIGO IPORRE COCA"]
for nombre in integrantes:
    tk.Label(sidebar, text=f"• {nombre}", fg="white",
             bg="#252525", font=("Arial", 11)).pack(anchor="w", padx=20, pady=2)
```

- Contenedor Lado a Lado (container_area): Es el núcleo visual que combina la referencia visual del robot (lbl_robot) con la matriz de botones, optimizando el espacio de la ventana.

```python
# --- CONTENEDOR LADO A LADO ---
container_area = tk.Frame(main_frame, bg="#1a1a1a")
container_area.pack(pady=20)

# Columna Imagen
col_img = tk.Frame(container_area, bg="#1a1a1a")
col_img.grid(row=0, column=0, padx=20)

try:
    img_robot = Image.open(r"C:\Users\IPORRE\Desktop\Prueba_Brazo_Comunicacion\brazo.png").resize((250, 300), Image.Resampling.LANCZOS)
    robot_tk = ImageTk.PhotoImage(img_robot)
    lbl_robot = tk.Label(col_img, image=robot_tk, bg="#1a1a1a")
    lbl_robot.image = robot_tk
    lbl_robot.pack()
except:
    tk.Label(col_img, text="Imagen [robot.png]", fg="white", bg="#333333", width=30, height=15).pack()

# Columna Botones
col_btns = tk.Frame(container_area, bg="#1a1a1a")
col_btns.grid(row=0, column=1)

def crear_boton_estilizado(f, texto, on, off, r, c):
    btn = tk.Button(f, text=texto, width=15, height=2,
                    bg="#333333", fg="white", activebackground="#deff9a",
                    font=("Arial", 10), relief="flat")
    btn.grid(row=r, column=c, padx=5, pady=5)
    btn.bind("<ButtonPress-1>", lambda e: enviar(on))
    btn.bind("<ButtonRelease-1>", lambda e: enviar(off))
```

c) LOGICA DEL CONTROL DINAMICO

El sistema utiliza una técnica llamada "Event Binding", que es el secreto detrás del movimiento fluido del brazo:

¿Cómo funcionan los botones?

ButtonPress-1: Al hacer clic, se envía instantáneamente una letra (ej. 'A'). El motor comienza a girar.

ButtonRelease-1: Al soltar el botón, se envía otra letra (ej. 'a'). El motor se detiene.

Resultado: El usuario tiene el control total. Si quieres un movimiento corto, solo haces un clic rápido. Si necesitas un giro largo, mantienes presionado el botón.

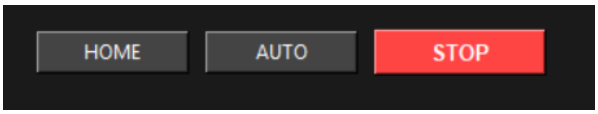
Automatización mediante Listas:

En lugar de programar cada botón manualmente, creamos una lista llamada controles. Esto hace que el código sea escalable: si mañana añaden un séptimo motor, solo añaden una línea a esa lista y el programa creará el botón automáticamente en la posición correcta (Fila, Columna).

```
controles = [
    ("Base IZQ", 'A', 'a', 0, 0), ("Base DER", 'B', 'b', 0, 1),
    ("Hombro SUB", 'C', 'c', 1, 0), ("Hombro BAJ", 'D', 'd', 1, 1),
    ("Codo SUB", 'E', 'e', 2, 0), ("Codo BAJ", 'F', 'f', 2, 1),
    ("Muñeca SUB", 'G', 'g', 3, 0), ("Muñeca BAJ", 'H', 'h', 3, 1),
    ("Giro IZQ", 'I', 'i', 4, 0), ("Giro DER", 'J', 'j', 4, 1),
    ("Cerrar Pinza", 'K', 'k', 5, 0), ("Abrir Pinza", 'L', 'l', 5, 1)
]

for ctrl in controles:
```

d). COMANDOS



Además de los movimientos manuales, hemos integrado tres botones de control de estado:

HOME: Envía el comando 'R'. El brazo debe volver a su posición inicial preprogramada.

AUTO: Envía el comando 'T'. Ejecuta una rutina de movimientos predefinida.

STOP: Envía el comando 'S'. Es el interruptor de emergencia. Su diseño (rojo, fuente negrita) está pensado para ser identificado rápidamente ante cualquier incidente.

```
    crear_boton_estilizado(col_btns, *ctrl)
# BOTONES DE SISTEMA (HOME, AUTO, STOP)
system_frame = tk.Frame(main_frame, bg="#1a1a1a")
system_frame.pack(pady=20) # Espacio debajo de la cuadrícula

tk.Button(system_frame, text="HOME", width=12, bg="#444444", fg="white",
          command=lambda: enviar('R')).pack(side="left", padx=5)

tk.Button(system_frame, text="AUTO", width=12, bg="#444444", fg="white",
          command=lambda: enviar('T')).pack(side="left", padx=5)

tk.Button(system_frame, text="STOP", width=12, bg="#FF4444", fg="white", font=("Arial", 10, "bold"),
          command=lambda: enviar('S')).pack(side="left", padx=5)
```

e). MODIFICACIONES A FUTURO

Si alguien más quiere aprender o modificar el programa, debe seguir estas reglas:

Rutas de Archivos: Al añadir imágenes (logo1.png, brazo.png), siempre usa rutas absolutas (ej: C:\Users\...) o asegúrate de que estén en la misma carpeta que el archivo .py para evitar errores de tipo No such file or directory.

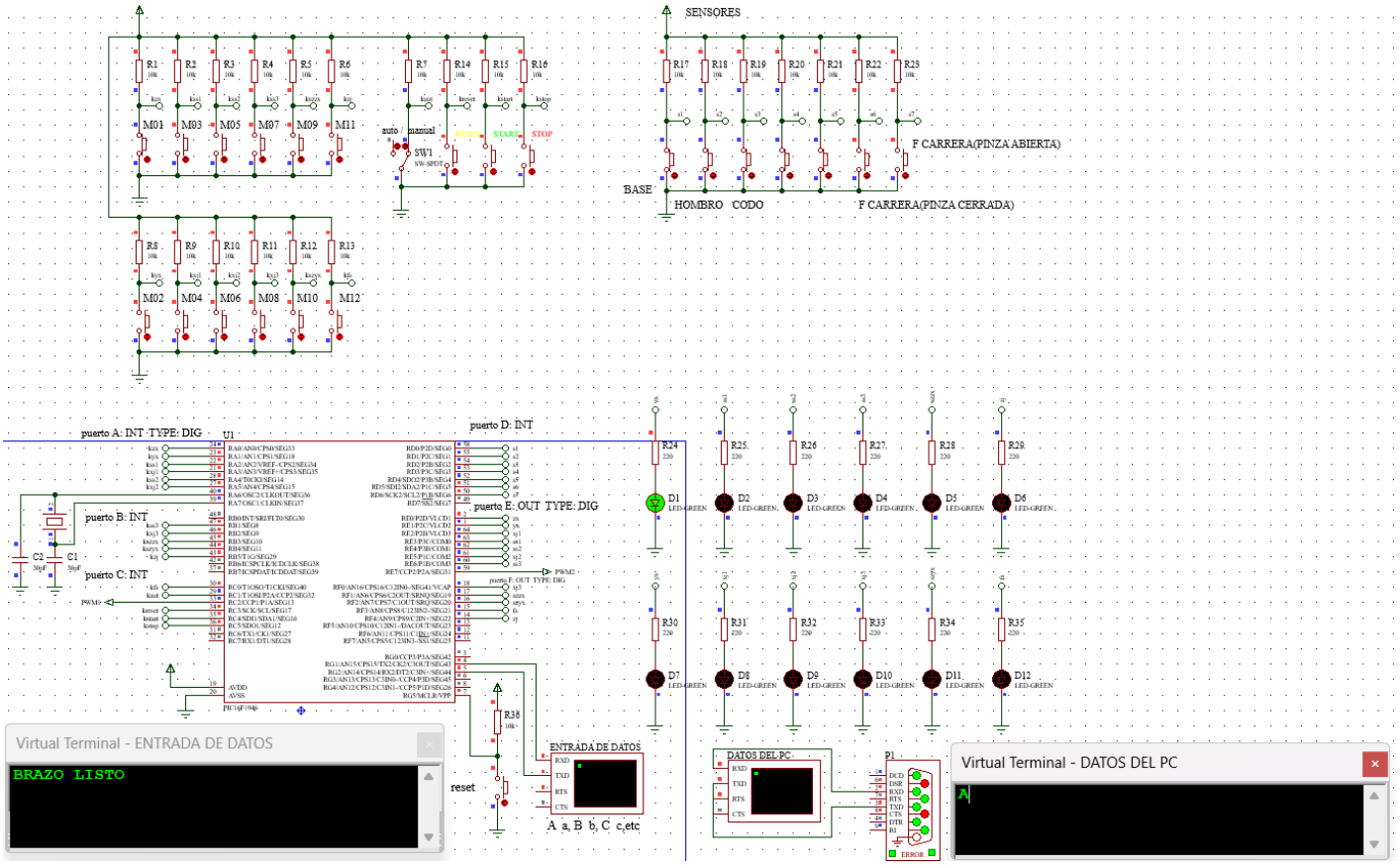
Escalabilidad de Botones: Si quieres cambiar el nombre de un botón, edita la lista controles. La función crear_boton_estilizado se encargará de mantener el diseño consistente.

Seguridad: Nunca modifiques la función cerrar_app(), ya que es la que protege al puerto serial de bloqueos permanentes.

2.4 Simulación y Supervisión del Sistema

Con el entorno de simulación configurado, se procedió a ejecutar diferentes pruebas de funcionamiento.

Durante esta etapa se evaluó el comportamiento del firmware ante distintos comandos de control. Se verificó que el microcontrolador fuera capaz de recibir información, procesarla adecuadamente y generar las acciones correspondientes según la lógica implementada.



La simulación permitió observar el comportamiento dinámico del sistema y analizar el flujo de datos entre los diferentes módulos del programa. Asimismo, se verificó la correcta ejecución de las rutinas encargadas de gestionar la comunicación serial.

2.4 Validación y Análisis de Resultados

La última etapa del proyecto consistió en verificar el correcto funcionamiento del sistema implementado.

Inicialmente, se presentaron inconvenientes durante la instalación del controlador (driver) del convertidor USB-Serial PL2303 en una computadora con sistema operativo Windows 11. El problema se debía a una incompatibilidad entre la versión más reciente del controlador y el dispositivo utilizado, lo que impedía el correcto reconocimiento y funcionamiento del puerto de comunicación serial.

Para solucionar este inconveniente, fue necesario desinstalar el controlador instalado automáticamente por el sistema operativo e instalar una versión más antigua y compatible del driver PL2303. Una vez realizada esta actualización, la computadora reconoció correctamente el dispositivo y se estableció la comunicación serial de manera satisfactoria, permitiendo la transmisión y recepción de datos entre la computadora y el microcontrolador.

Se compararon los resultados obtenidos durante la simulación con el comportamiento observado durante las pruebas físicas. Se comprobó que la lógica de funcionamiento validada en Proteus se mantenía durante la implementación real, evidenciando la utilidad de la simulación como herramienta de desarrollo.

Las pruebas realizadas permitieron verificar:

• Correcta recepción de comandos enviados desde la computadora.

• Correcta transmisión de datos mediante UART.

• Funcionamiento estable de la comunicación serial.

• Respuesta adecuada del sistema ante diferentes órdenes de control.

• Integración satisfactoria entre hardware y software.

Durante el proceso también se identificaron diversas dificultades relacionadas con la configuración de la comunicación serial, las cuales fueron corregidas mediante ajustes en el firmware y en los parámetros de comunicación utilizados por la interfaz de supervisión.

Finalmente, se concluyó que la metodología basada en simulación previa permitió reducir significativamente el tiempo de depuración y facilitar la implementación física del sistema.

Los resultados obtenidos demostraron la importancia de las herramientas de simulación y supervisión en el desarrollo de sistemas automatizados, permitiendo validar estrategias de control antes de su implementación definitiva y reduciendo la probabilidad de errores durante la puesta en funcionamiento.

3. RESULTADOS OBTENIDOS

Los resultados alcanzados durante el desarrollo del proyecto fueron satisfactorios.

En la etapa de simulación se logró verificar el correcto funcionamiento del firmware y de la comunicación serial.

En la etapa de implementación física se consiguió establecer una comunicación estable entre la computadora y el microcontrolador.

Finalmente, la interfaz desarrollada en Python permitió controlar el brazo robótico desde la computadora, validando la integración entre software y hardware.

Los objetivos principales relacionados con simulación, comunicación y supervisión fueron cumplidos exitosamente.

4. ANÁLISIS DE RESULTADOS

La utilización de Proteus permitió reducir significativamente el tiempo necesario para depurar errores de programación antes de trabajar con hardware real.

Asimismo, la comunicación serial implementada demostró ser una solución sencilla y efectiva para conectar la computadora con el sistema embebido.

La interfaz desarrollada en Python proporcionó una capa de supervisión que facilitó el control del sistema y mejoró la experiencia del usuario.

Los resultados obtenidos demuestran que la combinación de simulación, sistemas embebidos y herramientas de supervisión constituye una metodología efectiva para el desarrollo de sistemas automatizados.

5. LIMITACIONES DEL PROYECTO

Durante el desarrollo se identificaron algunas limitaciones:

• No se implementó realimentación mediante sensores continuos.

• El sistema opera principalmente mediante control por comandos.

• Algunas funciones avanzadas del brazo robótico permanecen en desarrollo.

• La simulación no reproduce completamente todas las condiciones presentes en el sistema físico.

CONCLUSIONES

La simulación realizada en Proteus permitió validar el firmware del sistema antes de su implementación física, reduciendo riesgos y tiempos de desarrollo.

La comunicación serial implementada mediante el adaptador PL2303HX permitió establecer un enlace confiable entre la computadora y el microcontrolador PIC16F1946.

La interfaz HMI desarrollada en Python facilitó la supervisión y el control del brazo robótico, integrando los diferentes componentes del sistema en una solución funcional.

Finalmente, el proyecto permitió demostrar la importancia de las herramientas de simulación, comunicación y supervisión en el desarrollo de sistemas automatizados modernos.

- ***Propuestas de mejora***

• Implementar sensores de posición para control en lazo cerrado.

• Incorporar monitoreo en tiempo real de variables del sistema.

• Desarrollar secuencias automáticas de movimiento.

• Implementar almacenamiento de trayectorias.

• Incorporar protocolos de comunicación industrial.

• Ampliar las capacidades de supervisión de la interfaz HMI.

11. BIBLIOGRAFÍA

[1] Microchip Technology Inc. PIC16F1946 Datasheet.

[2] Labcenter Electronics. Proteus Design Suite User Guide.

[3] Python Software Foundation. Python Documentation.